Title Date:
Page No:

Calculator Application: **a) Using with and without R objects on console.**

i)With R

```
print("Welcome to calculator(with R)")
operator <- readline("Enter operator (+, -, *, /): ")
num1 <- as.numeric(readline("Enter first number: "))
num2 <- as.numeric(readline("Enter second number: "))
if(operator == "+"){
result <- num1 + num2
print(paste("Result= ", result))
} else if(operator == "-"){
result <- num1 - num2
print(paste("Result= ", result))
} else if(operator == "*"){
result <- num1 * num2
print(paste("Result= ", result))
} else if(operator == "/"){
if(num2 != 0) {
Result <- num1 / num2
print(paste("Result=", result))
} else {
print("Error: Division by zero")
}
} else {

print("Invalid operator")
}
```

Output:

```
[1] "Welcome to calculator(with R)"
Enter operator (+, -, *, /): +
Enter first number: 6
Enter second number: 2
[1] "Result= 8";
```

Title Date:
Page No:

ii)Without R

Program:

```
print("Welcome to calculator(without R objects)")
cat("Enter first number: ")
num1 <;- as.numeric(readline())
cat("Enter operator(+,-,*,/)";)
op<-readline()
cat("Enter second number: ";)
num2 <- as.numeric(readline())
if(op == '+'){
cat("Result:",num1+num2,"\n")
} else if(op == '-'){
cat("Result:",num1-num2,"\n")
} else if(op == '*'){
cat("Result:",num1*num2,"\n")
} else if(op == '/'){
if(num2 != 0) {
cat("Result:",num1/num2,"\n")
} else {
print("Error: Division by zero")
}
} else {
print("Invalid operator")
}
```

Output:

```
[1] "Welcome to calculator(without R obects)";
Enter first number: 2
Enter operator(+,-,*,/): +
Enter second number: 3
Result: 5
```

Title

Date:
Page No:

b) Using mathematical functions on console.

```
num1<-as.numeric(readline("Enter first number:"))
num2<-as.numeric(readline("Enter Second number:"))
abs(num1)
round(num1,3)
ceiling(num1)
floor(num1)
sin(num2)
cos(num2)
tan(num1)
log(num2)
exp(num2)
factorial(num2)
choose(5,2)
sign(num1)
```

Output:

```
Enter first number:25
Enter Second number:30
abs(num1)   [1] 25
round(num1,3)   [1] 25
ceiling(num1)   [1] 25
floor(num1)     [1] 25
sin(num2)      [1] -0.9880316
cos(num2)     [1] 0.1542514
tan(num1)        [1] -0.1335264
log(num2)       [1] 3.401197
exp(num2)    [1] 1.068647e+13
factorial(num2)     [1] 2.652529e+32
choose(5,2)       [1] 10
sign(num1)        [1] 1
```

Title Date:
Page No:

c) Write an R script, to create R objects for calculator application and save in a specified location in disk.

```
op<-readline("Enter an operator(+,-,*,/):")
num1<-as.numeric(readline("Enter first number: "))
num2<;-as.numeric(readline("Enter second number: "))
if(op=='+'){
res<-num1+num2
}else if(op=='-'){
res<;-num1-num2
}else if(op=='*'){
res<-num1*num2
}else if(op=='/'){
res<-num1/num2
}else{
res<-NA
cat("Invalid Operator")
}
Calculator_data<-list(
number1=num1,
number2=num2,
operation=op,
result=res
)
save_path<-"C:/Users/ADMIN/Documents/L8/R/1c.R"
save(Calculator_data,file=save_path)
cat("data save successfully to",save_path,"\n")
```

Output:

```
Enter an operator(+,-,*,/):+
Enter first number: 10
Enter second number: 20
data d=save successfully to C:/Users/ADMIN/Documents/K8/R/1c.R
```

Title Date:
Page No:

d) R Packages: tidyr, ggplot2, ggraph, glue. shiny.

1)tidyr

```
library(tidyr)
data<-data.frame(
name=c(“Alice”,”Bob”),math=c(80,90),science=c(85,88)
)
long_data<-pivot_longer(data,cols=c(math,science),names_to”;subject”,values_to”score”)
print(long_data)
```

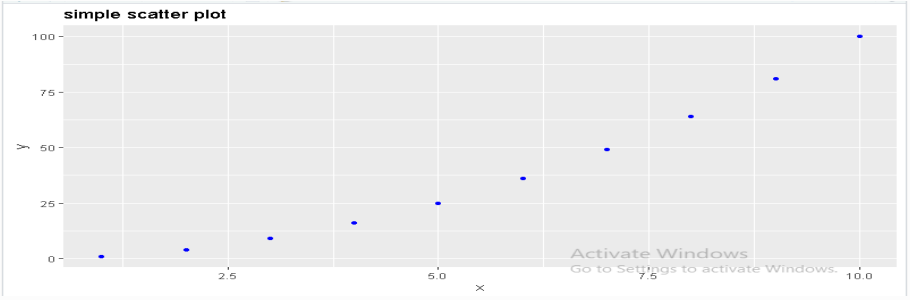
Output:

```
# A tibble: 4 × 3
name subject score
&lt;chr&gt; &lt;chr&gt; &lt;dbl&gt;
1 Alice math 80
2 Alice science 85
3 Bob math 90
4 Bob science 88
```

2)ggplot2

```
library(ggplot2)
df<-data.frame(x=1:10,y=(1:10)^2)
p<-ggplot(df,aes(x=x,y=y))+
geom_point(color=”blue”)+labs(title=”simple scatter plot”)
print(p)
```

Output

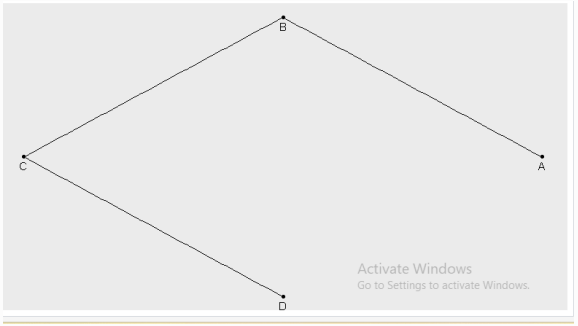


Title Date:
Page No:

3)ggraph

```
library(ggraph)
library(igraph)
graph <- graph_from_edgelist(
  cbind(c("A", "B", "C"), c("B", "C", "D"))
)
p <- ggraph(graph, layout = "circle") +
  geom_edge_link() +
  geom_node_point() +
  geom_node_text(aes(label = name), vjust = 1.5)
print(p)
```

Output



4)glue

```
library(glue)
name<-"Alice"
score<-95
message<-glue("Hello {name},your score is {score}!")
print(message)
```

Output

Hello Alice,your score is 95!

5)shiny

```
library(shiny)
ui <- fluidPage(
  titlePanel("Simple Shiny App"),
```

Title

Date:
Page No:

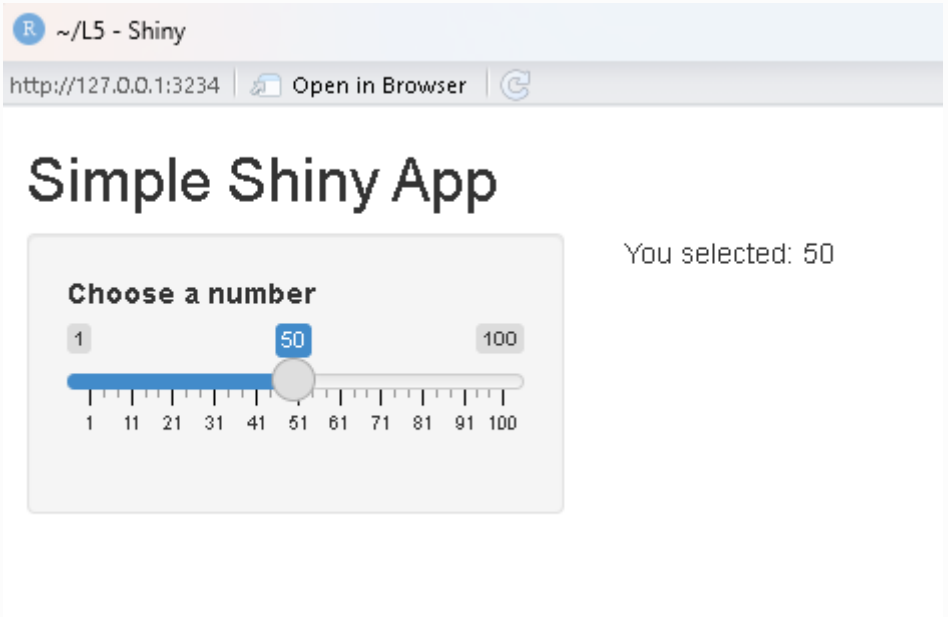
```
sidebarLayout(
   sidebarPanel(
     sliderInput("num", "Choose a number", 1, 100, 50)
   ),

   mainPanel(
     textOutput("result")
   )
 )
)
server <- function(input, output) {

  output$result <- renderText({
   paste("You selected:", input$num)
  })
}

shinyApp(ui = ui, server = server)
```

Output



Title Date:
Page No:

Descriptive Statistics: a) Write an R script to find basic descriptive statistics using summary.

```
data(iris)
df <- iris
head(df)
summary(df)
```

Output

```
> head(df)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa

> summary(df)
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width          Species
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100   setosa    :50
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300   versicolor:50
 Median :5.800   Median :3.000   Median :4.350   Median :1.300   virginica :50
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
```

Title Date:
Page No:

b) Write an R script to find subset of dataset by using subset.

```
employee <- data.frame(
  EmpID = c(101, 102, 103, 104, 105),
  Name = c("John", "Sara", "David", "Meena", "Alex"),
  Department = c("HR", "IT", "Finance", "IT", "HR"),
  Salary = c(30000, 45000, 40000, 50000, 32000)
)
print(employee)
it_employees <- subset(employee, Department == "IT")
print(it_employees)
```

Output

```
> print(employee)
  EmpID  Name Department Salary
1   101  John         HR  30000
2   102  Sara         IT  45000
3   103 David    Finance  40000
4   104 Meena         IT  50000
5   105  Alex         HR  32000

> it_employees <- subset(employee, Department == "IT")

> print(it_employees)

  EmpID  Name Department Salary
2   102  Sara         IT  45000
4   104 Meena         IT  50000
```

Title Date:
Page No:

Reading and Writing Different Types of Datasets: a) Reading different types of data sets (.txt, .csv) from web and disk and writing in file in specific disk location.

```
data_txt <- read.table(
  "C:/Rdata/sample.txt",
  header = TRUE
)
print(data_txt)
data_csv <- read.csv("C:/Rdata/sample.csv")
print(data_csv)
txt_url <- "https://example.com/sample.txt"
web_txt <- read.table(txt_url, header = TRUE)
print(web_txt)
csv_url <- "https://example.com/sample.csv"
web_csv <- read.csv(csv_url)
print(web_csv)
write.table(
  data_txt,
  "C:/Rdata/output.txt",
  row.names = FALSE
)
write.csv(
  data_csv,
  "C:/Rdata/output.csv",
  row.names = FALSE
)
cat("Files read and written successfully.")
```

OUTPUT:

```
Name Age Marks
1 Ravi  20    85
2 Sita  21    90
3  Ram  19    78

Files read and written successfully.
```

Title Date:
Page No:

b) Reading Excel data sheet in R.

```
install.packages("readxl")
library(readxl)
data <- read_excel("C:/Rdata/students.xlsx")
print(data)
data_sheet <- read_excel(
  "C:/Rdata/students.xlsx",
  sheet = 1
)
print(data_sheet)
write.csv(
  data,
  "C:/Rdata/students_output.csv",
  row.names = FALSE
)
cat("Excel data read successfully.")
```

OUTPUT:

```
# A tibble: 3 × 3

  Name    Age Marks
  <chr> <dbl> <dbl>
1 Ravi     20    85
2 Sita     21    90
3 Ram      19    78

Excel data read successfully.
```

c) Reading XML dataset in R.

```
install.packages("XML")
library(XML)
xml_data <- xmlParse("C:/Rdata/students.xml")
root <- xmlRoot(xml_data)
print(root)
students <- getNodeSet(root, "//student")
for (student in students) {
  name <- xmlValue(student[["name"]])
  age <- xmlValue(student[["age"]])
  marks <- xmlValue(student[["marks"]])
  cat("Name:", name, "\n")
  cat("Age:", age, "\n")
  cat("Marks:", marks, "\n\n")
}
```

Title

Date:
Page No:

OUTPUT:

Name: Ravi
Age: 20
Marks: 85

Name: Sita
Age: 21
Marks: 90

Name: Ram
Age: 19
Marks: 78

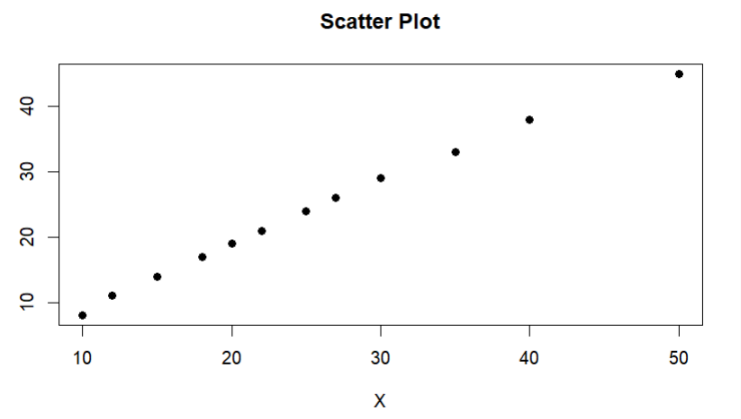
Title

Date:
Page No:

Visualizations: a) Find the data distributions using box and scatter plot.

```
x<-c(10,12,15,18,20,22,25,27,30,35,40,50)
y<-c(8,11,14,17,19,21,24,26,29,33,38,45)
boxplot(x,main="Box Plot",ylab="Values")
plot(x,y,main="Scatter Plot",xlab="X",ylab="Y",pch=19)
```

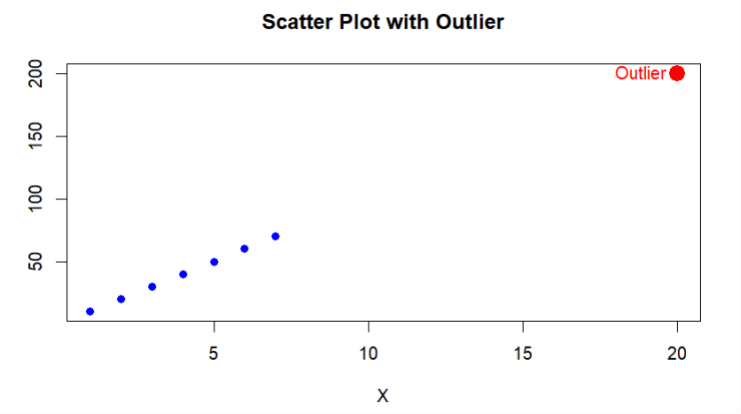
Output



b) Find the outliers using plot.

```
x <- c(1,2,3,4,5,6,7,20)
y <- c(10,20,30,40,50,60,70,200)
plot(x, y,
    main = "Scatter Plot with Outlier",
    xlab = "X", ylab = "Y",
    pch = 19, col = "blue")
points(20, 200, col = "red", pch = 19, cex = 2)
text(20, 200, labels = "Outlier", pos = 2, col = "red")
```

Output



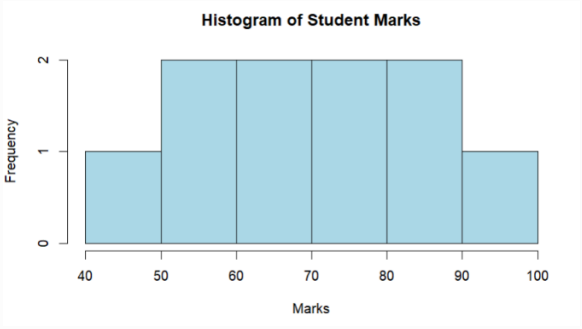
Title

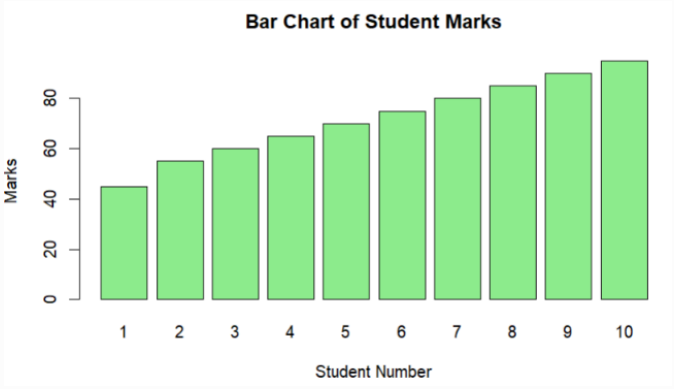
Date:
Page No:

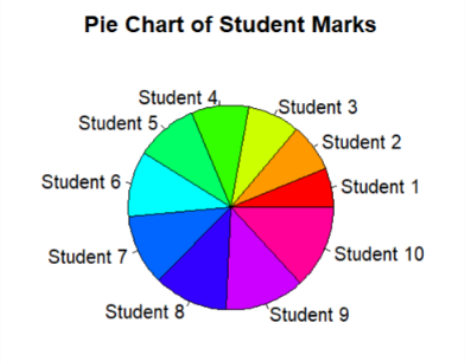
C) Plot the histogram, bar chart and pie chart on sample data

```
marks <- c(45, 55, 60, 65, 70, 75, 80, 85, 90, 95)
hist(marks, main = "Histogram of Student Marks", xlab = "Marks", ylab = "Frequency", col = "lightblue", border = "black")
barplot(marks, main = "Bar Chart of Student Marks", xlab = "Student Number", ylab = "Marks", names.arg = 1:10, col = "lightgreen", border = "black")
pie(marks, labels = paste("Student", 1:10), main = "Pie Chart of Student Marks", col = rainbow(length(marks)))
```

Output







Title Date:
Page:

Correlation and Covariance: a) Develop a program to find the correlation matrix on iris data.

```
data(iris)
iris_n<-iris[,1:4]
corr_matrix<-cor(iris_n)
print(corr_matrix)
```

Output

```
             Sepal.Length Sepal.Width Petal.Length Petal.Width
Sepal.Length    1.0000000  -0.1175698    0.8717538   0.8179411
Sepal.Width    -0.1175698   1.0000000   -0.4284401  -0.3661259
Petal.Length    0.8717538  -0.4284401    1.0000000   0.9628654
Petal.Width     0.8179411  -0.3661259    0.9628654   1.0000000
```

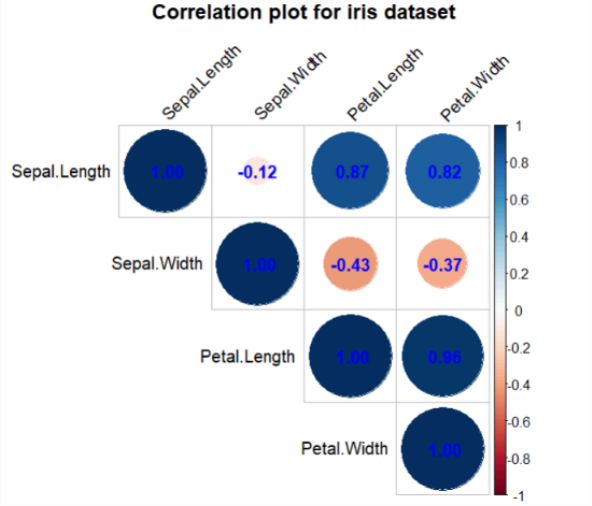
b) Plot the correlation plot on dataset and visualize giving an overview of relationships among data on iris data

```
install.packages("corrplot")
library(corrplot)
iris_n<-iris[,1:4]
corr_matrix<-cor(iris_n)
corrplot(corr_matrix,
        method="circle",
        type="upper",
        tl.col="black",
        tl.srt=45,
        addCoef.col="blue",
        title="Correlation plot for iris dataset",
        mar=c(0,0,2,0))
```

Output

Title

Date:
Page No:



c) Analysis of covariance: variance (ANOVA), if data have categorical variables on iris data.

```
data(iris)
anova_result<-aov(Sepal.Length~Species,data=iris)
summary(anova_result)
```

Output

```
             Df Sum Sq Mean Sq F value Pr(>F)
Species       2  63.21  31.606   119.3 <2e-16 ***
Residuals   147  38.96   0.265
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Title Date:
Page No:

Regression Model: a) Import data from web storage. Name the dataset and now do Logistic Regression to find out relation between variables that are affecting the admission of a student in an institute based on his or her GRE score, GPA obtained and rank of the student Also check if the model is fit or not. Require (foreign), require (MASS).

```
install.packages(c("foreign", "MASS"))
require(foreign)
require(MASS)
url <- "https://stats.idre.ucla.edu/stat/data/binary.csv"
admission <- read.csv(url)
admission$ rank <- factor(admission$ rank)
model <- glm(admit ~ gre + gpa + rank,
        data = admission, family = binomial)
summary(model)
anova(model, test = "Chisq")
```

Output

```
Call:
glm(formula = admit ~ gre + gpa + rank, family = binomial, data = admission)
Coefficients:
        Estimate Std. Error z value Pr(>|z|)
(Intercept) -3.989979   1.139951  -3.500 0.000465 ***
gre          0.002264   0.001094   2.070 0.038465 *
gpa          0.804038   0.331819   2.423 0.015388 *
rank2       -0.675443   0.316490  -2.134 0.032829 *
rank3       -1.340204   0.345306  -3.881 0.000104 ***
rank4       -1.551464   0.417832  -3.713 0.000205 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
(Dispersion parameter for binomial family taken to be 1)
   Null deviance: 499.98  on 399  degrees of freedom
Residual deviance: 458.52  on 394  degrees of freedom
AIC: 470.52
Number of Fisher Scoring iterations: 4
> anova(model, test = "Chisq")
Analysis of Deviance Table
Model: binomial, link: logit
Response: admit
Terms added sequentially (first to last)
```

Title Date:
Page No:

```
        Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                  399     499.98
gre   1  13.9204       398     486.06 0.0001907 ***
gpa   1   5.7122       397     480.34 0.0168478 *
rank  3  21.8265       394     458.52 7.088e-05 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

b) Apply regression Model to predict the data on any dataset.

```
data(mtcars)
model <- lm(mpg ~ wt, data = mtcars)
summary(model)
pred <- predict(model, newdata = mtcars)
print(head(data.frame(Actual = mtcars$mpg, Predicted = pred)))
rmse <- sqrt(mean((mtcars$mpg - pred)^2))
print(rmse)
plot(mtcars$wt, mtcars$mpg,
    main = "MPG VS WEIGHT",
    xlab = "Weight", ylab = "Mileage (MPG)",
    pch = 19, col = "blue")
abline(model, col = "red", lwd = 2)
```

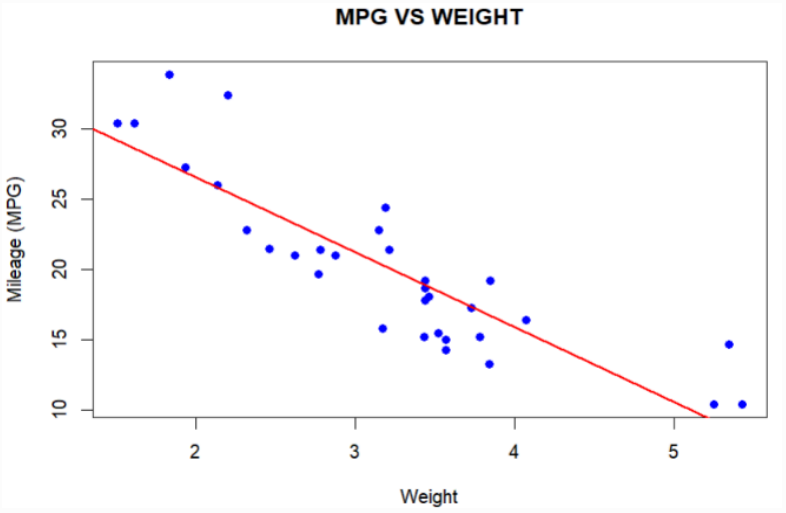
Output

```
> data(mtcars)
> model <- lm(mpg ~ wt, data = mtcars)
> summary(model)
Call:
lm(formula = mpg ~ wt, data = mtcars)
Residuals:
   Min      1Q  Median      3Q     Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.2851     1.8776  19.858  < 2e-16 ***
wt           -5.3445     0.5591  -9.559 1.29e-10 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Title

Date:
Page No:

```
Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528,  Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10
> pred <- predict(model, newdata = mtcars)
> print(head(data.frame(Actual = mtcars$mpg, Predicted = pred)))
                   Actual     Predicted
Mazda RX4           21.0      23.28261
Mazda RX4 Wag       21.0      21.91977
Datsun 710          22.8      24.88595
Hornet 4 Drive      21.4      20.10265
Hornet Sportabout   18.7      18.90014
Valiant             18.1      18.79325
> rmse <- sqrt(mean((mtcars$mpg - pred)^2))
> print(rmse)
[1] 2.949163
> plot(mtcars$wt, mtcars$mpg,
+     main = "MPG VS WEIGHT",
+     xlab = "Weight", ylab = "Mileage (MPG)",
+     pch = 19, col = "blue")
> abline(model, col = "red", lwd = 2)
```



Title

Date:
Page No:

Multiple Regression Model:

a) Develop Multiple Regression model for any dataset.

```
data(mtcars)
print(head(mtcars))
model <- lm(mpg ~ wt + hp + disp, data = mtcars)
print(summary(model))
predicted <- predict(model, mtcars)
plot(
  mtcars$mpg,
  predicted,
  main = "Actual vs Predicted MPG",
  xlab = "Actual MPG",
  ylab = "Predicted MPG",
  col = "blue",
  pch = 19
)
abline(a = 0, b = 1, col = "red", lwd = 2)
```

Output

```
> data(mtcars)
> print(head(mtcars))
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
> model <- lm(mpg ~ wt + hp + disp, data = mtcars)
> print(summary(model))
Call:
lm(formula = mpg ~ wt + hp + disp, data = mtcars)
Residuals:
   Min     1Q Median     3Q    Max
-3.891 -1.640 -0.172  1.061  5.861
```

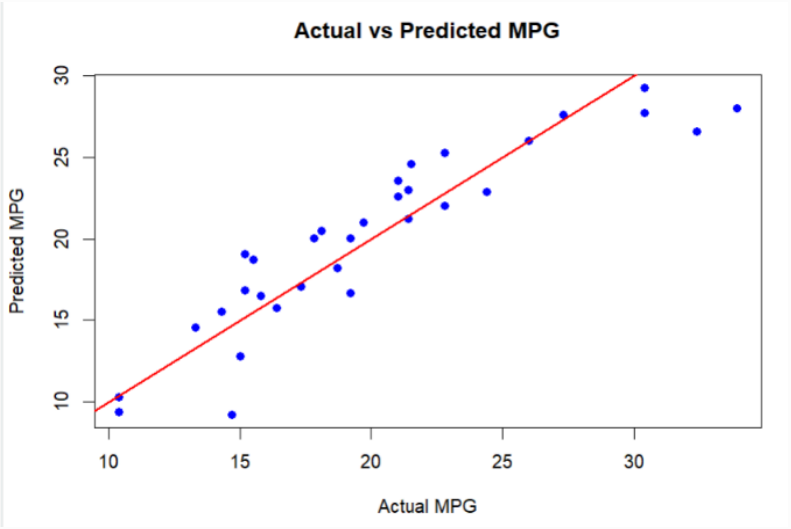
Title

Date:
Page No:

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.105505   2.110815  17.579  < 2e-16 ***
wt          -3.800891   1.066191  -3.565  0.00133 **
hp          -0.031157   0.011436  -2.724  0.01097 *
disp        -0.000937   0.010350  -0.091  0.92851
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.639 on 28 degrees of freedom
Multiple R-squared:  0.8268,  Adjusted R-squared:  0.8083
F-statistic: 44.57 on 3 and 28 DF,  p-value: 8.65e-11

> predicted <- predict(model, mtcars)
> plot(
+  mtcars$mpg,
+  predicted,
+  main = "Actual vs Predicted MPG",
+  xlab = "Actual MPG",
+  ylab = "Predicted MPG",
+  col = "blue",
+  pch = 19
+ )
> abline(a = 0, b = 1, col = "red", lwd = 2)
```



Title Date:
Page No:

b) Apply Multiple Regression Model to predict the data on any dataset.

```
data(iris)
set.seed(123)
i <- sample(1:nrow(iris), 0.8 * nrow(iris))
train <- iris[i, ]
test <- iris[-i, ]
model <- lm(Sepal.Length ~ Sepal.Width + Petal.Length + Petal.Width,
            data = train)
print(summary(model))
predicted <- predict(model, newdata = test)
print(data.frame(Actual = test$Sepal.Length,
                 Predicted = predicted))
rmse <- sqrt(mean((test$Sepal.Length - predicted)^2))
print(rmse)
plot(test$Sepal.Length, predicted,
     xlab = "Actual Sepal Length",
     ylab = "Predicted Sepal Length",
     main = "Actual vs Predicted",
     pch = 19, col = "blue")
abline(a = 0, b = 1, col = "red")
```

Output

```
> data(iris)
> set.seed(123)
> i <- sample(1:nrow(iris), 0.8 * nrow(iris))
> train <- iris[i, ]
> test <- iris[-i, ]
> model <- lm(Sepal.Length ~ Sepal.Width + Petal.Length + Petal.Width,
+             data = train)
> print(summary(model))
Call:
lm(formula = Sepal.Length ~ Sepal.Width + Petal.Length + Petal.Width,
   data = train)
Residuals:
     Min       1Q   Median       3Q      Max
-0.82577 -0.21712  0.02843  0.18999  0.85864
```

Title Date:
Page No:

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.89882    0.28429   6.679 8.74e-10 ***
Sepal.Width   0.63637    0.07606   8.367 1.52e-13 ***
Petal.Length  0.69826    0.06208  11.247  < 2e-16 ***
Petal.Width  -0.53303    0.13964  -3.817 0.000218 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

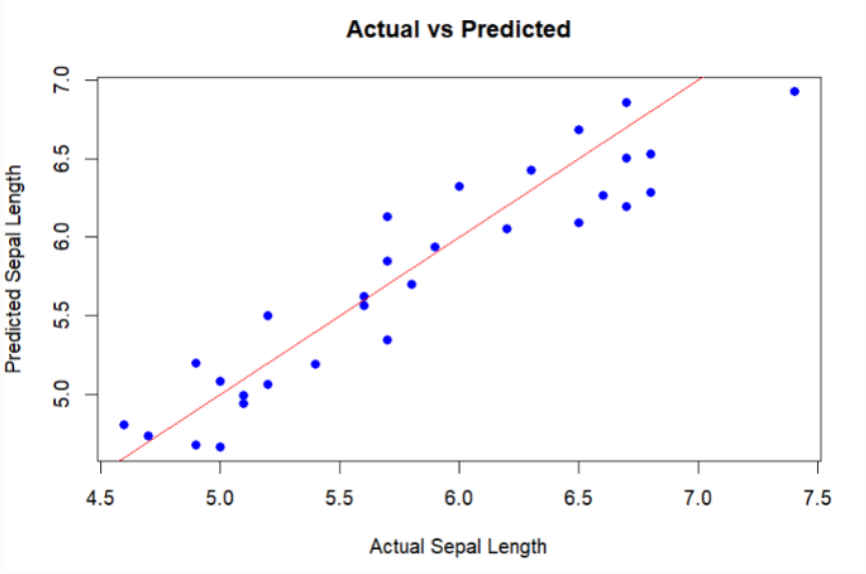
Residual standard error: 0.3258 on 116 degrees of freedom
Multiple R-squared:  0.856,    Adjusted R-squared:  0.8523
F-statistic: 229.9 on 3 and 116 DF,  p-value: < 2.2e-16

> predicted <- predict(model, newdata = test)
> print(data.frame(Actual = test$Sepal.Length,
+                  Predicted = predicted))
    Actual Predicted
1      5.1  4.997076
2      4.9  4.678890
3      4.7  4.736338
11     5.4  5.194177
18     5.1  4.943773
19     5.7  5.344163
28     5.2  5.066902
33     5.2  5.502028
36     5.0  4.666512
48     4.6  4.806165
55     6.5  6.093117
56     5.7  6.129897
57     6.3  6.427826
58     4.9  5.197342
59     6.6  6.263360
61     5.0  5.082446
62     5.9  5.941086
65     5.6  5.565098
66     6.7  6.197679
70     5.6  5.626634
77     6.8  6.286073
83     5.8  5.700605
84     6.0  6.325308
```

Title

Date:
Page No:

```
98     6.2  6.053881
100    5.7  5.850592
105    6.5  6.685185
113    6.8  6.529010
125    6.7  6.859574
131    7.4  6.927299
141    6.7  6.502564
> rmse <- sqrt(mean((test$Sepal.Length - predicted)^2))
> print(rmse)
[1] 0.2687939
> plot(test$Sepal.Length, predicted,
+      xlab = "Actual Sepal Length",
+      ylab = "Predicted Sepal Length",
+      main = "Actual vs Predicted",
+      pch = 19, col = "blue")
> abline(a = 0, b = 1, col = "red")
```



Title Date:
Page No:

Classification Model: a) Install relevant package for classification.

```
install.packages("class")
install.packages("e1071")
install.packages("rpart")
install.packages("caret")
library(class)
library(e1071)
library(rpart)
library(caret)
```

Output:

```
Loading required package: ggplot2
Attaching package: ‘ggplot2’
The following object is masked from ‘package:e1071’:
    element
Loading required package: lattice
```

b) Implement and evaluate the performance of various classifiers with different datasets.

```
install.packages(c("class", "rpart", "e1071"))
library(class)
library(rpart)
library(e1071)
data(iris)
set.seed(123)
i <- sample(1:nrow(iris), 0.8 * nrow(iris))
train <- iris[i, ]
test <- iris[-i, ]
evaluate <- function(actual, predicted) {
  cm <- table(Actual = actual, Predicted = predicted)
  print(cm)
  accuracy <- sum(diag(cm)) / sum(cm)
  print(paste("Accuracy:", round(accuracy * 100, 2), "%"))
}
x_train <- scale(train[, 1:4])
x_test <- scale(test[, 1:4],
                center = attr(x_train, "scaled:center"),
                scale = attr(x_train, "scaled:scale"))
knn_pred <- knn(x_train, x_test, train$Species, k = 3)
```

Title Date:
Page No:

```
cat("KNN Confusion Matrix and Accuracy:\n")
evaluate(test$Species, knn_pred)
tree <- rpart(Species ~ ., data = train, method = "class")
tree_pred <- predict(tree, test, type = "class")
cat("Decision Tree Confusion Matrix and Accuracy:\n")
evaluate(test$Species, tree_pred)
nb <- naiveBayes(Species ~ ., data = train)
nb_pred <- predict(nb, test)
cat("Naive Bayes Confusion Matrix and Accuracy:\n")
evaluate(test$Species, nb_pred)
```

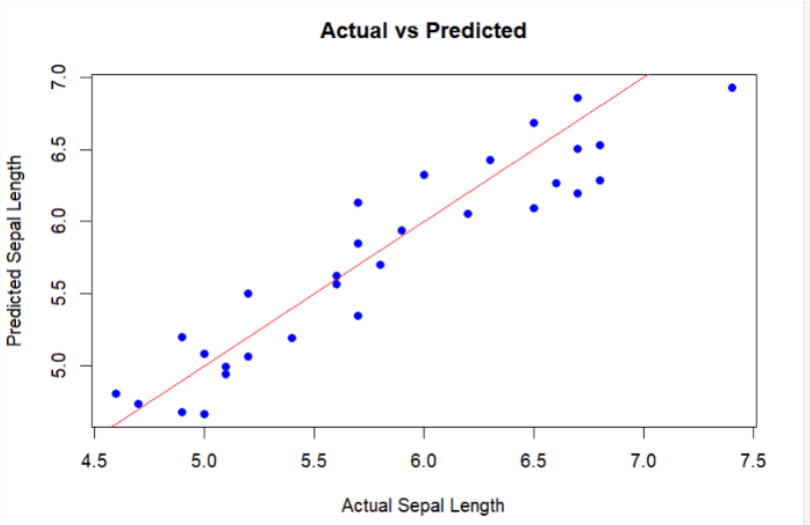
Output:

```
> library(class)
> library(rpart)
> library(e1071)
> data(iris)
> set.seed(123)
> i <- sample(1:nrow(iris), 0.8 * nrow(iris))
> train <- iris[i, ]
> test <- iris[-i, ]
> evaluate <- function(actual, predicted) {
> knn_pred <- knn(x_train, x_test, train$Species, k = 3)
> cat("KNN Confusion Matrix and Accuracy:\n")
KNN Confusion Matrix and Accuracy:
> evaluate(test$Species, knn_pred)
                Predicted
Actual      setosa versicolor virginica
  setosa        10          0         0
  versicolor     0         14         1
  virginica      0          0         5
[1] "Accuracy: 96.67 %"
> tree <- rpart(Species ~ ., data = train, method = "class")
> tree_pred <- predict(tree, test, type = "class")
> cat("Decision Tree Confusion Matrix and Accuracy:\n")
Decision Tree Confusion Matrix and Accuracy:
> evaluate(test$Species, tree_pred)
```

Title

Date:
Page No:

```
          Predicted
Actual      setosa versicolor virginica
  setosa        10          0         0
  versicolor     0         13         2
  virginica      0          0         5
[1] "Accuracy: 93.33 %"
> nb <- naiveBayes(Species ~ ., data = train)
> nb_pred <- predict(nb, test)
> cat("Naive Bayes Confusion Matrix and Accuracy:\n")
Naive Bayes Confusion Matrix and Accuracy:
> evaluate(test$Species, nb_pred)
          Predicted
Actual      setosa versicolor virginica
  setosa        10          0         0
  versicolor     0         15         0
  virginica      0          0         5
[1] "Accuracy: 100 %"
```



Title Date:
Page No:

Clustering Model:

a) Clustering algorithms for unsupervised classification.

```
install.packages("ggplot2")  # Run only once
library(ggplot2)
data(iris)
df <- iris[, c("Sepal.Length", "Petal.Length")]
df_scaled <- scale(df)
set.seed(123)
km <- kmeans(df_scaled, centers = 3, nstart = 25)
print(km$cluster)
print(km$centers)
df$Cluster <- as.factor(km$cluster)
head(df)
```

Output:

```
print(km$cluster)
  [1] 1 1 1
```

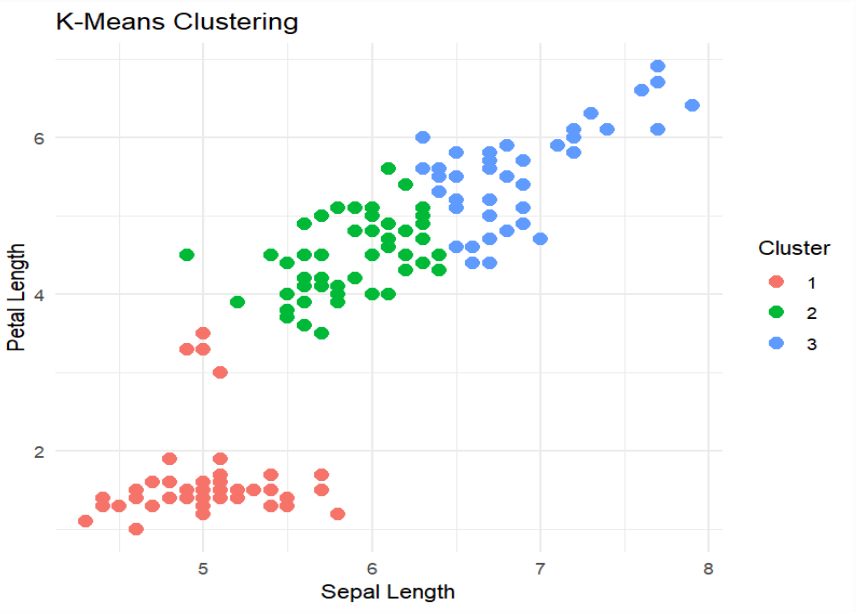
Title

Date:
Page No:

b) Plot the cluster data using R visualizations.

```
ggplot(df, aes(x = Sepal.Length,
               y = Petal.Length,
               color = Cluster)) +
  geom_point(size = 3) +
  labs(title = "K-Means Clustering",
       x = "Sepal Length",
       y = "Petal Length",
       color = "Cluster") +
  theme_minimal()
```

Output



Title

Date:
Page No:

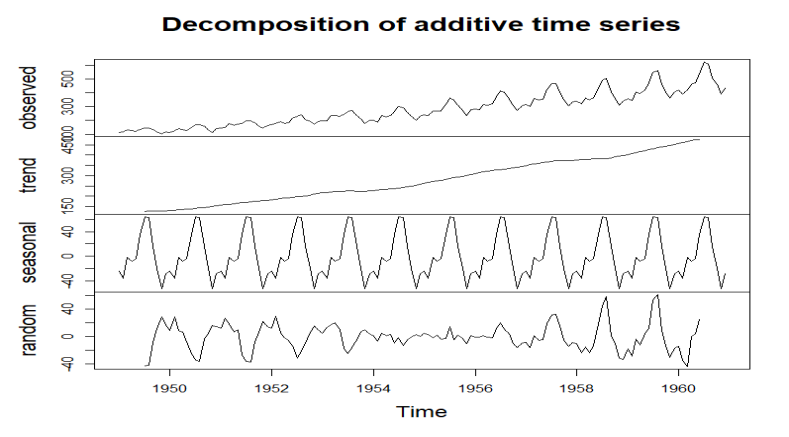
Time Series Model:

a) Plot the Time series data using R visualizations.

```
data(AirPassengers)
print(AirPassengers)
plot(AirPassengers,
    main = "Monthly Air Passengers",
    xlab = "Year",
    ylab = "Number of Passengers",
    col = "blue",
    lwd = 2)
grid()
decomp <- decompose(AirPassengers)
plot(decomp)
```

Output:

```
> print(AirPassengers)
    Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1949 112 118 132 129 121 135 148 148 136 119 104 118
1950 115 126 141 135 125 149 170 170 158 133 114 140
1951 145 150 178 163 172 178 199 199 184 162 146 166
1952 171 180 193 181 183 218 230 242 209 191 172 194
1953 196 196 236 235 229 243 264 272 237 211 180 201
1954 204 188 235 227 234 264 302 293 259 229 203 229
1955 242 233 267 269 270 315 364 347 312 274 237 278
1956 284 277 317 313 318 374 413 405 355 306 271 306
1957 315 301 356 348 355 422 465 467 404 347 305 336
1958 340 318 362 348 363 435 491 505 404 359 310 337
1959 360 342 406 396 420 472 548 559 463 407 362 405
1960 417 391 419 461 472 535 622 606 508 461 390 432
```



Title

Date:
Page No:

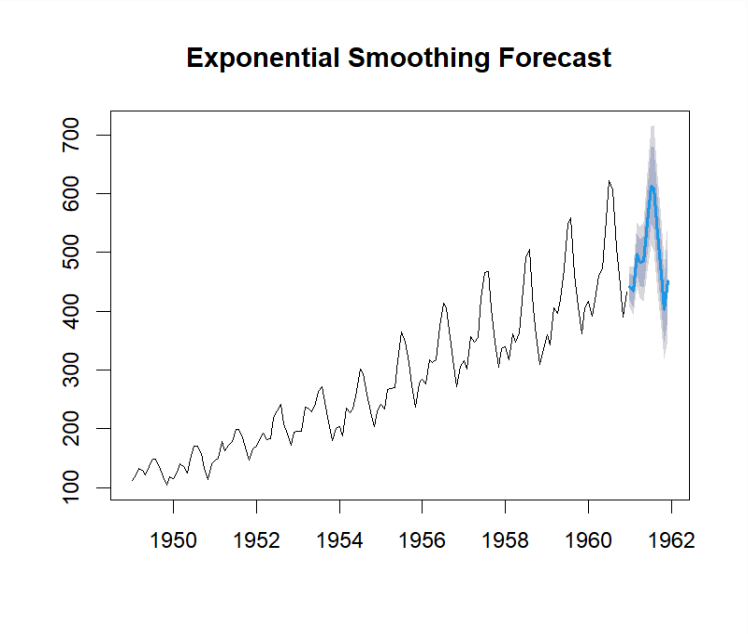
b) Time series Forecasting using Exponential smoothing, ARIMA models.

1. Exponential Smoothing (Holt-Winters method)

```
install.packages("forecast")  # Run only once
library(forecast)
ets_model <- ets(AirPassengers)
ets_forecast <- forecast(ets_model, h = 12)
print(ets_forecast)
plot(ets_forecast,
    main = "Exponential Smoothing Forecast")
```

Output:

```
> print(ets_forecast)
         Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
Jan 1961       441.8018 419.6256 463.9780 407.8863 475.7174
Feb 1961       434.1186 407.1668 461.0704 392.8994 475.3379
Mar 1961       496.6300 460.6291 532.6310 441.5714 551.6887
Apr 1961       483.2375 443.6210 522.8539 422.6493 543.8256
```



Title

Date:
Page No:

2. ARIMA model

```
arima_model <- auto.arima(AirPassengers)
summary(arima_model)
arima_forecast <- forecast(arima_model, h = 12)
print(arima_forecast)
plot(arima_forecast,
    main = "ARIMA Forecast")
```

Output

```
Series: AirPassengers
ARIMA(2,1,1)(0,1,0)[12]
Coefficients:
         ar1     ar2      ma1
      0.5960  0.2143  -0.9819
s.e.  0.0888  0.0880   0.0292
sigma^2 = 132.3:  log likelihood = -504.92
AIC=1017.85   AICc=1018.17   BIC=1029.35
Training set error measures:
                    ME     RMSE      MAE       MPE     MAPE      MASE
Training set 1.3423 10.84619 7.86754 0.420698 2.800458 0.245628
Training set -0.00124847
> print(arima_forecast)
         Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
Jan 1961       445.6349 430.8903 460.3795 423.0851 468.1847
Feb 1961       420.3950 403.0907 437.6993 393.9304 446.8596
Mar 1961       449.1983 429.7726 468.6240 419.4892 478.9074
Apr 1961       491.8399 471.0270 512.6529 460.0092 523.6707
May 1961       503.3945 481.5559 525.2330 469.9953 536.7937
Jun 1961       566.8624 544.2637 589.4612 532.3007 601.4242
```

